

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Y Greeshma Reddy	Reg. No.:	192424146
Section / Batch:	B Tech-AI&DS	Date:	14-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q1. A software company must schedule multiple client meetings in limited time slots, ensuring maximum number of non-overlapping meetings. Analyze how the Activity Selection problem can be solved using a Greedy approach. Design a solution by explaining the selection strategy based on finishing times. Justify why the greedy method provides an optimal solution. Suggest suitable tools or programming techniques for implementation. Analyze time complexity and interpret the results in terms of efficient scheduling.

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y Greeshma Reddy

RUBRICS FOR CALCULATION**Industry Problem Solving Task**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				

Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Given a set of activities with start and finish times, select the maximum number of non-overlapping activities using the Greedy Activity Selection algorithm.

1. Understanding of Industry Problem

- Clearly understands the real-world requirement of scheduling multiple client meetings within limited working hours.
- Identifies that meetings should be selected in such a way that no two selected meetings overlap.
- Recognizes that the main objective is to schedule the maximum possible number of meetings.
- Understands that each meeting has a start time and finish time, which are the important parameters for scheduling.
- Relates the Activity Selection problem to practical applications such as meeting scheduling, conference-room allocation, appointment management, and resource utilization.

2. Application of Engineering Concepts

- Applies the Greedy Algorithm to solve the scheduling problem efficiently.
- Uses the concept of selecting the meeting with the earliest finishing time.
- Understands that selecting an activity that finishes earlier leaves more time available for subsequent activities.
- Applies algorithm design concepts such as sorting, selection criteria, constraints, and optimization.

- Demonstrates knowledge of time complexity and algorithm efficiency.
- Uses the greedy-choice property to obtain an optimal solution for the Activity Selection problem.

3. Solution Design

- The solution first accepts the number of meetings and their corresponding start and finish times.
- All meetings are sorted according to their finishing times in ascending order.
- The first meeting with the earliest finishing time is selected.
- The remaining meetings are checked one by one.
- A meeting is selected only when its start time is greater than or equal to the finish time of the previously selected meeting.
- This process continues until all meetings have been considered.
- The final selected meetings represent the maximum number of non-overlapping meetings.
- The design is simple, systematic, and avoids unnecessary calculations.
- The greedy strategy is optimal because choosing the earliest finishing compatible meeting leaves the maximum remaining time for other meetings.

4. Use of Tools

- Python can be used to implement the Activity Selection algorithm.
- Python's built-in `sort()` function can be used to arrange activities according to their finishing times.
- A suitable IDE, code editor, or online compiler can be used for implementation and execution.
- Test cases can be provided through standard input to verify the correctness of the program.
- Debugging tools can be used to identify errors and verify the selected activities.
- Basic programming constructs such as loops, conditional statements, lists, and sorting are sufficient for implementation.
- The solution does not require complex external libraries, making it easy to implement and test.

5. Analysis

- The activities are first sorted based on their finishing times.
- Sorting takes $O(n \log n)$ time.
- Selecting compatible activities after sorting takes $O(n)$ time.
- Therefore, the overall time complexity is $O(n \log n)$.
- The space requirement is generally $O(n)$ for storing the activities and selected results.

- The greedy approach is much more efficient than checking every possible combination of meetings.
- The results demonstrate that maximum meeting utilization can be achieved while avoiding scheduling conflicts.
- The method is suitable for large scheduling problems because of its efficient performance.
- The algorithm provides an optimal solution specifically because of the properties of the Activity Selection problem.

IMPLEMENTATION

The screenshot shows the SIMATS Online Programming Lab interface. The page title is "SIMATS Online Programming Lab" with a sub-header "DAA PROGRAMMING". The user is identified as "YANAMALA GREESHMA REDDY". The question is titled "Activity Selection - Scheduling Optimization" and is worth 100 marks. The question text states: "A software company must schedule multiple client meetings in limited time slots, ensuring maximum number of non-overlapping meetings. Analyze how the Activity Selection problem can be solved using a Greedy approach. Design a solution by explaining the selection strategy based on finishing times. Justify why the greedy method provides an optimal solution. Suggest suitable tools or programming techniques for implementation. Analyze time complexity and interpret the results in terms of efficient scheduling." The user's code is displayed in a dark-themed editor, showing a Python implementation of the greedy algorithm. The code reads input, sorts activities by finish time, and selects non-overlapping activities. The input is:


```
6
1 2
3 4
6 2
9 3
8 1
3 5
```

 The output is:


```
5
Selected activities:
1 2
3 4
6 2
```

 The interface also includes a "Run" button, a "Save" button, and a "Submit" button. The bottom status bar shows the temperature as 36°C, mostly sunny, and the time as 13:38 on 14-08-2026.

6. Professionalism

- Provides a clear and logically structured solution to the industry problem.
- Uses meaningful variable names and follows proper programming practices.
- Maintains readable and understandable code structure.
- Clearly explains the greedy selection strategy and why it works.
- Includes appropriate test cases to validate the implementation.
- Reports the time and space complexity of the solution.
- Presents the results in a professional manner that can be easily understood by technical and non-technical users.
- Demonstrates the ability to connect engineering concepts, algorithm design, programming implementation, and real-world scheduling requirements.